

Laboratorio 3: "Arquitectura Lambda para el monitoreo y análisis de la calidad del aire"

Ignacio Ampuero - Engel Falcón - Santiago Herrera - Matías Huerta

Profesora Ximena Prado

04 octubre, 2026

Índice

Introducción	3
Objetivos.....	3
Tabla comparativa de arquitecturas	4
Arquitectura elegida y justificación	5
Materiales y entorno	6
Herramientas.....	7
Diseño del Pipeline.....	11
Ejecución paso a paso	12
Resolución de problemas y tips.....	19
Conclusiones	21
Bibliografía	21
Anexo A. Glosario de términos.....	23
Anexo B. Evidencias	25

Introducción

El presente Laboratorio 03 implementa una solución basada en la Arquitectura Lambda para abordar el problema transversal del curso: el monitoreo de la calidad del aire en cuatro ciudades chilenas.

Objetivos

Objetivo general

Construir y demostrar un pipeline Lambda funcional de punta a punta sobre calidad del aire, reproducible en otro dispositivo siguiendo los comandos exactos de este informe.

Objetivos específicos

1. Descargar la foto de datos una sola vez y dejarla en disco con manifiesto y hash verificables.
2. Ingerir los eventos en un topic de Kafka y comprobar de forma automática que llegaron completos y sin duplicados.
3. Procesar el tema casi en vivo con Spark Structured Streaming y una ventana de sesión sobre la hora de lectura.
4. Procesar el histórico completo en modo lote y producir las vistas consolidadas de la semana.
5. Exponer las dos capas en una capa de servicio sobre SQLite con el cruce entre velocidad y lotes.
6. Dejar evidencia de todo el flujo, con comandos exactos, salidas reales y hashes que permitan repetir la corrida.
7. Dejar lista una composición de Docker con el broker de Kafka y su verificador, como alternativa opcional al arranque local.
8. Comparar la arquitectura Lambda con Kappa y Lakehouse sobre la misma pregunta, en la tabla comparativa que se entrega por separado.

Tabla comparativa de arquitecturas

	Arquitectura Lambda	Arquitectura Kappa	Data Lake / Data Lakehouse
Latencia	Combina procesamiento por lotes para el historial y una capa de velocidad para entregar resultados	Orientada a sistemas que requieren procesar los datos de forma continua y con baja latencia	Depende de la implementación, puede manejar cargas batch periódicas o integrar streaming para obtener indicadores con baja latencia
Complejidad	Alta, porque incrementa la complejidad técnica al requerir la implementación y el mantenimiento de dos flujos de procesamiento diferentes	Media a alta, debido al dominio técnico requerido en streaming y gestión de eventos continuos	Es variable, depende del ecosistema tecnológico utilizado, la escala de los datos y el nivel de gobierno implementado
Reprocesamiento	La capa batch puede recalculer los resultados a partir de los datos originales que se encuentran almacenados	Permite ejecutar análisis históricos o corregir errores mediante el reprocesamiento de eventos almacenados anteriormente en el registro	Facilita la conservación de datos originales para reprocesarlos posteriormente cuando cambian las reglas de negocio o surgen nuevas necesidades
Tolerancia a fallos	Proporciona tolerancia a fallos gracias a que la capa batch permite recalculer y sobrescribir los resultados usando los datos originales	Exige implementar un monitoreo constante del flujo y contar con mecanismos de recuperación ante fallos y control de errores	En el caso del Lakehouse, incorpora características de Data Warehouse para fortalecer la trazabilidad, calidad y gobierno de los datos estructurados y no estructurados

Costo	Exige mayor capacidad técnica y de infraestructura para monitorear, mantener y sincronizar dos rutas separadas	Simplifica la arquitectura al evitar dos rutas, pero el reprocesamiento de historiales masivos demanda alta capacidad de infraestructura	Escala eficientemente ante el aumento de volumen y en muchos casos reduce costos frente a plataformas de almacenamiento tradicionales
Consistencia	La capa batch genera resultados consistentes, pero se requieren mecanismos de coordinación para evitar inconsistencias entre las dos capas	Reduce la duplicación de transformaciones y reglas de negocio al centralizar todo en un flujo único	El enfoque Lakehouse incorpora mecanismos como el control de versiones y transacciones que fortalecen la consistencia de los datos
Casos de uso	Sistemas que necesitan análisis histórico exhaustivo y, simultáneamente, resultados de baja latencia	Sistemas enfocados en el monitoreo, la respuesta inmediata y los eventos continuos	Plataformas integrales enfocadas en ciencia de datos, inteligencia artificial, analítica, BI y centralización de información masiva

Arquitectura elegida y justificación

La solución implementada se basa en la Arquitectura Lambda, estructurada en tres niveles que operan sobre un flujo de ingesta unificado: una capa de velocidad para resultados inmediatos, una capa de lotes para el procesamiento consolidado y una capa de servicio para resolver las consultas.

El pilar de este diseño es que el dato ingresa una única vez, se publica en un tópic de Kafka y desde allí se bifurca. De esta manera, ambos caminos de procesamiento se

alimentan exactamente del mismo origen, optimizando los recursos y evitando repetir el crítico proceso de ingesta.

La elección de Lambda se fundamenta en la necesidad del laboratorio de responder a la misma interrogante sobre la calidad del aire manejando dos ritmos distintos de forma simultánea. Además, el diseño permite que las dos ramas converjan en la capa de servicio, convirtiendo a la arquitectura en un mecanismo de autovalidación técnica: los resultados de ambos caminos deben coincidir matemáticamente, probando la integridad del sistema.

Materiales y entorno

Componente	Versión	Por qué esa
Python	3.12.10	venv propia dentro del repositorio
Temurin JDK	17.0.20.1	Spark 3.5 solo acepta Java 8, 11 o 17
PySpark	3.5.9	empaqueta Hadoop 3.3.4, que tiene winutils publicado
pandas	2.3.3	PySpark 4.x no soporta del todo pandas 3 en adelante
winutils	3.3.5	el binario nativo que Windows necesita para escribir
Apache Kafka	4.1.2	broker en modo KRaft, sin ZooKeeper
kafka-python	3.0.11	productor y consumidor ligeros para la ingesta
SQLite	el de Python 3.12	viene de fábrica, sin servidor ni instalación aparte
PyYAML	6.0.2	lee el compose en el verificador estático
Docker	20.10.4 o posterior	opcional en el enunciado, solo para la composición

Herramientas

En esta parte se explicará de forma adecuada cada herramienta utilizada dentro del proyecto en el mismo orden, qué es, cómo funciona dentro de este laboratorio, con qué comandos se usa y qué fragmentos del código la manejan

Apache Kafka

Kafka es una plataforma de eventos distribuida donde los productores publican mensajes en tópicos y los consumidores los leen, y el broker es el servidor que recibe, guarda y reparte esos mensajes en particiones.

Acá corre un broker local en el puerto 9092 con un heap de 256 megabytes y sin ZooKeeper, porque desde Kafka 4.0 el modo ZooKeeper fue eliminado y el broker se gobierna a sí mismo en modo Kraft.

El dato entra por un solo topic, siendo este lab03.eventos, la clave radica en que cada mensaje es la ciudad para que las lecturas de un mismo lugar queden juntas en la misma partición, y la confirmación de escritura es acks=1, que es el punto medio entre la velocidad y la garantía.

Utilizando Powershell:

```
& scripts\start_kafka.ps1
```

```
& .venv\Scripts\python.exe lambda\01-ingesta\02_productor.py --lista --mensajes 8
```

```
& .venv\Scripts\python.exe lambda\01-ingesta\03_verificar.py
```

```
& scripts\stop_kafka.ps1
```

El arranque espera la línea exacta que dice que el servidor terminó de encender y responde con 'El broker arranco bien' y 'puerto 9092 responde True', y el verificador se puede correr cuando sea necesario.

El productor se abre en '02_productor.py:314' con la confirma y los serializadores que trae la librería, y manda cada evento con su hora de emisión recién anotada en el momento exacto de la salida.

```
return KafkaProducer(
```

```
    bootstrap_servers=config.KAFKA_BROKER,
```

```
    acks=1,
```

```
    linger_ms=0,
```

```
value_serializer=JsonSerializer(),  
key_serializer=DefaultSerializer(),  
)  
...  
futuro = productor.send(  
arg.topic, key=evento["ciudad"], value=evento)  
futuro.get(timeout=30)
```

El ‘send’ devuelve un futuro y el ‘get(timeout=30)’ espera la confirmación antes de contar el mensaje como enviado, así que los 672 eventos enviados y 0 fallidos del resumen son confirmaciones reales y no intenciones.

Apache Spark y PySpark

Spark es un motor unificado de procesamiento que sirve igual para lotes y flujos, y PySpark es su API de Python, donde la misma consulta se escribe sobre una tabla estática o sobre una columna de eventos que se amplía sin parar.

El mismo topic se lee de dos maneras distintas, en lote se recorre completo de una vez con ‘spark.read’, y en streaming se escribe con ‘readStream’ tomando como máximo cien mensajes por lote, con marcas de agua de cuatro horas y ventana de sesión de dos horas.

Utilizando Python:

Este es 06_lotes.py:85

```
spark.read.format("kafka")  
.option("startingOffsets", "earliest")  
.option("endingOffsets", "latest")  
.load()
```

Este de acá es streaming, 04_velocidad.py:79

```
spark.readStream.format("kafka")  
.option("startingOffsets", "earliest")
```

```
.option("maxOffsetsPerTrigger", config.MAXOFF_SETS_PER_LOTE)  
.load()
```

```
eventos.withWatermark("hora_lectura", MARCA_AGUA)  
.groupBy(F.col("ciudad"),  
F.session_window("hora_lectura", ESPACIO_SESION))  
.agg(F.count(F.lit(1)).alias("lecturas"), ...)
```

Con esto el streaming cierra en el disparador availableNow, que procesa todo lo disponible al arrancar y termina solo, y el lote no lleva disparador porque recorre el tema entero en una sola corrida batch.

```
& .venv\Scripts\python.exe lambda\02-velocidad\04_velocidad.py --limpiar  
& .venv\Scripts\python.exe lambda\03-lotes\06_lotes.py
```

Python y la venv

Python es el lenguaje del laboratorio y la venv es una carpeta con su propio intérprete y sus propias librerías dentro del repositorio, de modo que el proyecto no depende de lo que tenga instalado el dispositivo.

La versión fijada es 3.12.10 con las librerías pineadas en el requirements.txt, y la instalación siempre se hace con la ruta absoluta del intérprete de la venv, dado que en la pc en que se planifico la creación, tenía instalada otra versión de python.

Usaremos los siguientes comandos para verificar un par de nimiedades:

```
& .venv\Scripts\python.exe --version  
& .venv\Scripts\python.exe -m pip install -r requirements.txt  
& .venv\Scripts\python.exe -W error -m py_compile lambda\01-ingesta\01_descargar.py  
& .venv\Scripts\python.exe src\comun\smoke_test.py
```

La compilación con '-W error' convierte cualquier aviso en error para que una deprecación no pase desapercibida, y la prueba de humo cierra con siete de siete pruebas, al final la línea SMOKE TEST: OK.

Además, cada script imprime al empezar un encabezado de trazabilidad con el momento en UTC, commit, el comando exacto y las versiones, generado por el script de evidencia.py:115, así que un log suelto se explica bastante bien con esta metodología.

SQLite

SQLite es una base de datos completa que vive en un solo archivo y viene con Python de fábrica, no pide un servidor o una instalación previa o aparte, la base entera viaja junto al código.

Acá se usa en la capa de servicio como un punto único de consulta, en el archivo aire.db que se borra y se rehace en cada ejecución para que nunca se mezclen corridas viejas con corridas nuevas.

Con el siguiente comando:

```
& .venv\Scripts\python.exe lambda\04-servicio\07_servicio.py
```

Dentro encontramos esto de acá entre las lineas 131 y 156:

```
campos = “, “.join(f”{columna}{tipo}”  
for columna, tipo in columnas.items())  
con.execute(f”CREATE TABLE IF NOT EXISTS {nombre} ({campos})”)  
insertar = f”INSERT INTO {nombre} ({‘, ‘.join(columnas)}) ” \  
    f”VALUES ({‘, ‘.join(‘?’ for _ in columnas)})”  
...  
)
```

Las tablas se declaran con sus tipos antes de cargar, y si el encabezado de un CSV no coincide con lo declarado la ejecución se corta en lugar de meter números viejos en silencio, y sobre esas tablas corre un left join del cruce que junta horas del batch con lecturas de episodios.

Docker

Docker empaqueta servicios en contenedores reproducibles, y en este laboratorio se usa solo como alternativa para levantar el mismo broker de Kafka sin instalarlo a mano.

La composición es mínima, dado que es un solo servicio con la imagen oficial de Apache, el puerto 9092 publicado y un volumen nombrado para los datos.

Git

Git es el sistema de control de versiones que guarda el historial del proyecto en commits, y aquí cumple dos trabajos, trazar qué código produjo el log y servir para evidencias de avances.

El encabezado de cada script no imprime el último commit que tocó ese archivo sino el HEAD del repositorio entero en ese momento, porque el log lo produce todo el código junto, incluidos los módulos compartidos.

Diseño del Pipeline

La ingesta es una sola y empieza con la imagen o fuente de datos, que trae lecturas horarias de PM2.5, PM10 y NO2 en cuatro ciudades, Santiago, Mendoza, Valparaíso y Puerto Montt, y el umbral de 25 microgramos por metro cúbico que marca la hora contaminada viene del límite diario que sugiere la Organización Mundial de la Salud.

El productor recorre la imagen de datos en orden creciente de hora de lectura y manda cada uno de los eventos con su hora de lectura y su hora de emisión por separado, la primera dice cuándo se midió el aire y la segunda prueba que el mensaje salió en vivo.

Después de ahí el dato se bifurca, y la frontera es el topic, porque lo que ya llegó está del lado de las capas de procesamiento y lo que sigue llegando sigue acumulándose en la cola.

La capa de velocidad lee el tema con Spark Structured Streaming, toma como máximo cien mensajes por lote para que la ejecución se vea repartida y las sesiones se puedan observar creciendo de uno a otro, y agrupa por ciudad con una ventana de sesión de dos horas sobre la hora de lectura.

La ventana de sesión encaja con el problema porque un episodio de combinación es una ráfaga de lecturas altas separadas de las siguientes por un hueco de calma, la sesión junta lo que viene pegado y deja lo demás afuera sin tener que adivinar el ancho de una ventana fija.

La marca de agua va en cuatro horas, el doble del espacio de sesión a propósito, porque si quedará más cerca de la ventana podría cerrar una sesión que todavía estaba abierta, y con ella Spark acota la memoria que usa para el estado.

El resultado se escribe en Parquet dentro de la carpeta de salidas en modo completo, de manera que cada lote vuelve a mandar todas las sesiones que siguen vivas, y el

checkpoint de la ejecución vive en la carpeta checkpoints, con la opción '--limpiar' para borrar el checkpoint y Parquet cuando se quiera arrancar desde cero.

La capa de lotes espera a que la cola esté completa, recorre el tema de punta a punta, y escribe cada vista dos veces en Parquet para que Spark la relea sin parsear y en CSV para que se pueda abrir en cualquier plantilla.

Las vistas se sobrescriben en cada corrida, así que no hace falta ningún interruptor de limpieza para arrancar de nuevo, y al final cada vista recibe su SHA-256 con campos estables, de modo que dos corridas sobre el mismo tema dan los mismos hashes.

La capa de servicio guarda todo en SQLite, en el archivo aire.db que se borra y se rehace en cada ejecución, y expone cuatro consultas, el resumen de la semana por ciudad con horas sobre umbral y porcentaje, las ciudades con sus totales, los episodios del streaming y el cruce final.

El cruce pone en la misma fila las horas contaminadas que cuenta del batch y las que suman los episodios del streaming, y las dos tienen que dar el mismo número porque es el mismo dato contado por caminos diferentes, uno desde los mensajes y otro desde las vistas.

Ese cruce es la comprobación final de la arquitectura lambda, si las dos columnas cuadran las dos ramas procesaron exactamente lo mismo y la capa de servicio puede responder sin favoritos.

Ejecución paso a paso

instalación del entorno y prueba de humo

```
.\scripts\bootstrap.ps1
```

Prepara el entorno en el que estaremos trabajando: java 17, el .venv de python 3.12, Pyspark, winutils y apache kafka. Al preparar el entorno corre una prueba de humo para comprobar que estos funcionen juntos.

- Imprime un bloque verificación con la línea "SMOKE TEST, OK" nos confirma que el entorno quedo y está listo para la ejecución de los comandos posteriores.

```
Verificacion
SMOKE TEST: OK
log en evidencias\logs\00_smoke_test.log
```

arranque del bróker de kafka

.\scripts\start_kafka.ps1

Levanta el broker apache kafka 4.1.2 en un segundo plano, se confirma el arranque y se comprueba que el puerto 9092 acepta conexiones. Al estar vivo, avisa y no levanta uno más.

- Imprime “el broker arranco bien”
- “puerto 9092 responde true”, el broker dentro del puerto nos escucha. Por lo tanto, es una buena señal para continuar el procedimiento.

```
El broker arranco bien
puerto 9092 responde True
PS C:\Users\herre\BigDataLab03> |
```

descarga de los datos

& .venv\Scripts\python.exe lambda\01-ingesta\01_descargar.py aire_horario

Es el único script que usa la red para así descargar desde Open-Meteo una semana cerrada de lecturas horarias de PM2,5 PM10 Y NO2 para santiago, Mendoza, Valparaíso y puerto Montt, estas las guarda en un JSON y crea un manifiesto con la URL, el rango y el SHA-256.

- Imprime un encabezado con: fecha UTC, commit, comando, python, pyspark y plataforma.
- Fuente aire_horario, rango 2026-09-27 a 2026-10-03 y la URL de la API usada.
- Ruta del archivo guardado, tamaño y SHA-256.

```
\01_descargar.py aire_horario
01_descargar.py 2026-10-04T16:01:28+00:00
commit 9f04ab5
comando lambda\01-ingesta\01_descargar.py aire_horario
python 3.12.10 en C:\Users\herre\BigDataLab03\.venv\Scripts\python.exe
pyspark 3.5.9
plataforma Windows-11-10.0.26200-SP0

Fuente aire_horario
Rango 2026-09-27 a 2026-10-03
URL https://air-quality-api.open-meteo.com/v1/air-quality?latitude=-33.45,-32.99,-33.05,-41.47&longitude=-70.67,-68.85,-71.62,-72.94&hourly=pm2_5,pm10,nitrogen_dioxide&timezone=America%2FSantiago&start_date=2026-09-27&end_date=2026-10-03

Descargando

Guardado C:\Users\herre\BigDataLab03\datos\raw\aire_horario_2026-09-27_2026-10-03.json
Tamano 47823 bytes
SHA-256 1558c512df4e9a746c1e39fa952f535c691229aac165533ad065d54df634a4a7
Manifiesto C:\Users\herre\BigDataLab03\datos\raw\aire_horario_manifiesto.json
Log C:\Users\herre\BigDataLab03\evidencias\logs\01_descarga.log
PS C:\Users\herre\BigDataLab03> |
```

```
& .venv\Scripts\python.exe lambda\01-ingesta\02_productor.py --mensajes 672 --segundos 0.02 --limpiar
```

Este comando envía a Kafka los 672 datos de calidad del aire, uno cada 0.02 segundos para simular datos que lleguen en vivo, esto empezando con el broker limpio, y al final imprime cuantos se enviaron, cuantos fallaron y cuantas alertas hubo (111)

- Imprime los enviados que fueron 672 y fallidos siendo 0. Esto demuestra que llegaron todos los mensajes.
- Alertas 111: lecturas sobre el umbral
- Topic lab03.eventos y broker localhost:9092
- Duración 15.2 s y ritmo de 44.34 mensajes por segundo.

```
Enviados    672
Fallidos    0
Alertas     111
Topic       lab03.eventos
Broker      localhost:9092
Duracion    15.2 s
Ritmo       44.34 msg/s
```

verificación de la ingesta

```
& .venv\Scripts\python.exe lambda\01-ingesta\03_verificar.py
```

Lee todo el topic desde el principio, sin grupo de consumo, revisa que cada evento traiga los campos completos y calcula los campos estables, para comprobar que la ingesta es reproducible.

- Imprime los 672 recibidos y las 111 alertas, iguales a lo enviado por el productor.
- Ciudades con 168 lecturas cada una (mendoza, puerto montt, santiago, valparaiso).
- La primera y última hora.
- El SHA-256 con campo estable y la lista de campos que entran al hash.

```

PS C:\Users\herre\BigDataLab03> & .venv\Scripts\python.exe lambda\01-ingesta\03_verificar.py
03_verificar.py 2026-10-04T16:04:58+00:00
commit 9f04ab5
comando lambda\01-ingesta\03_verificar.py
python 3.12.10 en C:\Users\herre\BigDataLab03\.venv\Scripts\python.exe
pyspark 3.5.9
plataforma Windows-11-10.0.26200-SP0

Topic lab03.eventos
Broker localhost:9092

Recibidos 672
Alertas 111
Ciudades mendoza=168, puerto_montt=168, santiago=168, valparaiso=168
Primer hora 2026-09-27T00:00
Ultima hora 2026-10-03T23:00

SHA-256 estable c046073b98901122d0ba093cb9cee8f3a25c2fe6a93fa5e4bb7c434b4c5323d8
Campos evento_id, ciudad, hora_lectura, pm2_5, pm10, nitrogen_dioxide, umbral, supera_umbral
Fecha corrida 2026-10-04T16:05:04+00:00
  
```

capa de velocidad: streaming con spark

& .venv\Scripts\python.exe lambda\02-velocidad\04_velocidad.py --limpiar

Spark structured streaming lee el topic casi en vivo, se queda con las lecturas sobre el umbral y las agrupa pr ciudad en episodios con una ventana de sesión de 2 horas, marca de agua de 4 horas y un máximo de 100 mensajes por lote. El resultado se escribe en parquet --limpiar borra el checkpoint y el parquet de la corrida anterior.

- Imprime 7 líneas, del lote 001 al 007, con las sesiones vivas de cada lote, con el acumulado que llega hasta 28.
- Lotes 7: la semana se procesó en 7 lotes en total.
- 28 filas con episodios repetidos entre lotes, porque el modo completo reenvía las sesiones abiertas en cada lote.

```

Lote 001 sesiones vivas      2 acumulado      2
Lote 002 sesiones vivas      2 acumulado      4
Lote 003 sesiones vivas      4 acumulado      8
Lote 004 sesiones vivas      4 acumulado     12
Lote 005 sesiones vivas      5 acumulado     17
Lote 006 sesiones vivas      5 acumulado     22
Lote 007 sesiones vivas      6 acumulado     28

Lotes      7
Filas      28 con episodios repetidos entre lotes

Filas en disco 28
Episodios      6
  
```

resumen de la capa de velocidad

& .venv\Scripts\python.exe lambda\02-velocidad\05_resumen_velocidad.py

Como el nodo completo repite cada episodio en cada lote, este script se queda con la versión más avanzada de cada uno, escribe episodios.csv, genera el grafico velocidad_episodios.png y calcula el SHA-256 de los episodios.

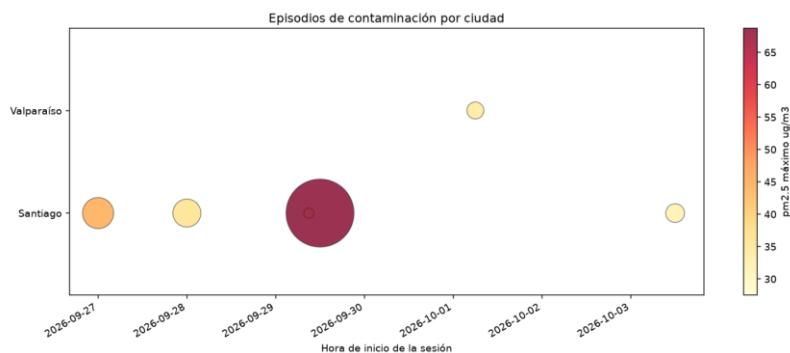
- Episodios 6: santiago = 5 y Valparaíso = 1
- Primera 2026-09-27 00:00 y ultima 2026-10-03 18:00
- Mas horas: santiago, 74 horas seguidas, desde 2026-09-29 12:00
- Mas picado: santiago, 68,7 u/m3 en 2026-09-29 12:00
- SHA-256 de episodios, firma de reproducibilidad de esta capa

```

Episodios 6
Ciudades santiago=5, valparaiso=1
Primera 2026-09-27 00:00:00
Ultima 2026-10-03 18:00:00

Mas horas Santiago 2026-09-29 12:00:00 74 horas seguidas
Mas picado Santiago 68.7 ug/m3 en 2026-09-29 12:00:00

SHA-256 episodios befb30473fcb54e617c392857566ecbc6e506c3d5663c90fc36df577c
53e1bf6
  
```



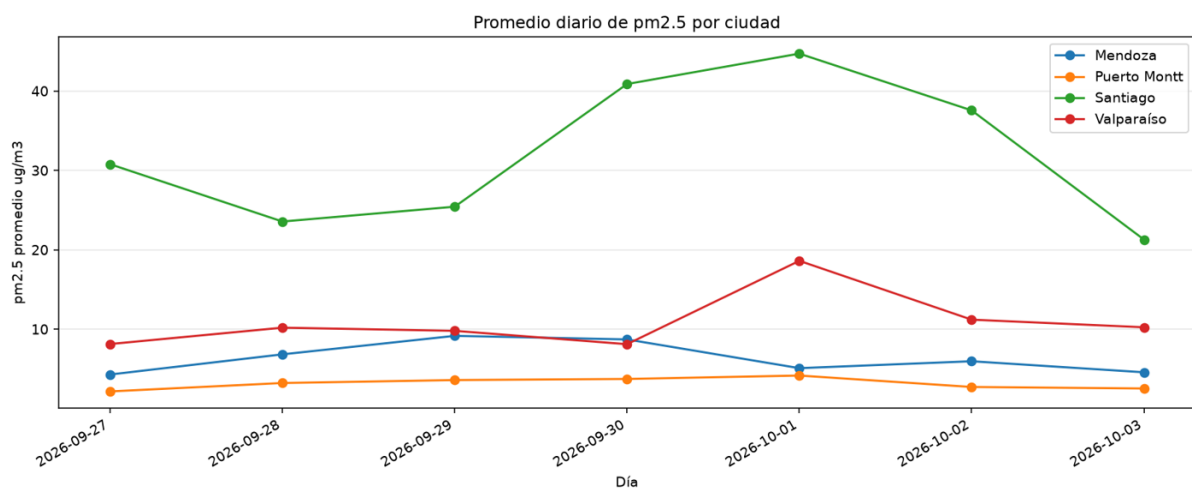
capa de lotes

& .venv\Scripts\python.exe lambda\03-lotes\06_lotes.py

Lee el topic completo de una sola vez, como corrida batch, sin filtrar nada, y construye dos vistas consolidadas: una diaria (ciudad por día) y otra por ciudad. Cada vista se guarda en parquet y CSV con su propio SHA-256 y se genera el grafico lotes_diario.png.

- Imprime los eventos 672 y alertas 111.
- Diario 28 filas (4 ciudades x 7 días) ciudades 4 filas.
- Horas umbral 111, igual que las alertas: la capa de lotes cuenta lo mismo que la ingesta.

```
Eventos      672
Alertas      111
Diario       28 filas
Ciudades     4 filas
Horas umbral 111 igual que las alertas
```



capa de servicio: consultas y cruce de las dos capas

& .venv\Scripts\python.exe lambda\04-servicio\07_servicio.py

Carga los 3 CVS (diario, ciudades y episodios) es una base SQLite, aire, db, que se rehace en cada corrida y ejecuta cuatro consultas.

- Imprime las filas cargadas: 28 diario, 4 ciudades y 6 episodios.
- Resumen por ciudad
- Días más contaminados

- Episodios del streaming por ciudad

```

diario      28 filas desde diario.csv
ciudades    4 filas desde ciudades.csv
episodios    6 filas desde episodios.csv

Resumen por ciudad
ciudad      lecturas  horas umbral  pct umbral  pm2.5 prom
Santiago    168        107         63.69      32.02
Valparaíso  168         4          2.38      10.89
Mendoza     168         0           0.0        6.36
Puerto Montt 168         0           0.0        3.15

Dias mas contaminados
ciudad      dia        pm2.5 prom  pm10 prom
Santiago    2026-10-01  44.7       47.04
Santiago    2026-09-30  40.87      45.14
Santiago    2026-10-02  37.55      38.79

Episodios del streaming por ciudad
ciudad      episodios  horas ventana  lecturas contaminadas
Santiago    5          113.0         107
Valparaíso  1           5.0           4

Cruce de las dos capas
ciudad      horas batch  lecturas episodios  episodios
Santiago    107          107             5
Valparaíso  4            4             1
Mendoza     0            0             0
Puerto Montt 0            0             0

Horas batch      111
Lecturas episodios 111
El cruce          coinciden
  
```

cierre del bróker de kafka

.\scripts\stop_kafka.ps1

Detiene el proceso de kafka desde la misma consola y comprueba que el puerto 9092 quedo libre.

- Imprime “deteniendo el broker de kafka”
- “el broker se detuvo, el puerto 9092 quedo libre”

```

PS C:\Users\herre\BigDataLab03> .\scripts\stop_kafka.ps1
Deteniendo el broker de Kafka
El broker se detuvo, el puerto 9092 quedo libre
  
```

verificación del Docker-compose

& .venv\Scripts\python.exe docker\verificar_compose.py

Revisa de forma estática el archivo docker\docker-compose.yml usando PyYAML, sin necesidad de Docker instalado. Comprueba lo que Kafka necesita para arrancar en modo KRaft: imagen, puerto, sol de broker y controlador, claves de KRaft, replicación, volumen y healthcheck.

- Imprime 13 líneas OK, una por comprobación: el YAML se lee, existe el kafka, la imagen coincide con la instalación local, el puerto 9092 está publicado, el nodo es broker y controlador, las claves de KRaft están completas, la replicación está en uno, el volumen está declarado y montado, y el healthcheck consulta el broker.
- Total 13 comprobaciones, 0 fallos

```
OK      el archivo parsea como diccionario
OK      el servicio kafka existe
OK      la imagen es apache/kafka igual a la instalación local
OK      el puerto 9092 del host queda publicado
OK      el nodo hace de broker y de controlador a la vez
OK      el nodo tiene identificador
OK      el cliente escucha en el mismo broker que usa el proyecto
OK      las claves de KRaft están todas
OK      los factores de replicación están en uno
OK      el volumen de datos está declarado
OK      el contenedor monta ese volumen donde escribe Kafka
OK      las bitácoras del broker apuntan al volumen
OK      el healthcheck consulta al broker con kafka-topics
Total   13 comprobaciones, 0 fallos
```

Resolución de problemas y tips

1. El pip del PATH pertenece a un 3.13 mientras que python abre el 3.12, así que los paquetes se instalan siempre con la ruta absoluta de la venv, & .venv\Scripts\python.exe -m pip install -r requirements.txt, y nunca con el pip suelto.
2. La pc trae un Java 20 de Oracle en el PATH que Spark arranca igual y después revienta, la solución es dejar que la configuración escriba JAVA_HOME apuntando al Temurin 17 antes de importar PySpark, y la prueba de humo confirma qué Java se usó.

3. Si Windows bloquea un script de la carpeta scripts se levanta el permiso con 'Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass', que vive solo mientras la terminal esté abierta y se borra al cerrarla, sin tocar la política de la máquina.
4. Si la escritura de Parquet falla con un error que parece una ruta mal escrita, en realidad faltan permisos de Windows y hay que revisar que winutils exista y que HADOOP_HOME apunte a su carpeta.
5. El 04_velocidad.py se debe correr con --limpiar cuando el tema se mandó desde cero, porque el checkpoint viejo le dice a Spark que ya leyó todo y la corrida nueva no procesa nada.
6. El 06_lotes.py no se debe correr a mitad de la producción del productor, las vistas se calculan sobre lo que haya en el tema en ese instante y sale una semana incompleta con todos los hashes cambiados.
7. No se deben convertir todos los avisos en error en los scripts que pasan un DataFrame a pandas, PySpark avisa por un método que pandas ya marcó como obsoleto y el script se cae sin que haya nada malo, la compilación con avisos en error sí aplica a los demás scripts.
8. Si el conector de Kafka no encuentra las clases es porque se pidió con sufijo 2.13 y Spark trae 2.12, hay que descargar el conector con el sufijo correcto.
9. Si el streaming revienta con 'Update output mode not supported for session window' no es la máquina, es que alguien cambió el modo de salida, la capa de velocidad va en modo completo.
10. Si al terminar Spark aparece 'ERROR ShutdownHookManager' sobre una carpeta temporal, es Windows que no puede borrar la carpeta mientras la JVM todavía tiene los jars abierta, el resumen ya se escribió antes y el error es inofensivo.
11. Si el 06_lotes.py dice 'El topic esta vacio, corre primero 02_productor.py' el productor no corrió o alguien limpió el tema después de él, se repite la secuencia completa desde el productor con --limpiar.
12. Si el renglón de cruce no da 111 o el servicio dice 'NO coinciden', el tema se leyó a mitad de la producción o las dos capas leyeron temas distintos, se vuelve a la secuencia completa antes de desconfiar de las vistas.
13. Si la sesión de Spark no arranca o se queda colgado, la máquina quedó sin memoria libre después de varias corridas seguidas, se espera a que se libere y se reintenta sin cambiar nada.
14. No se debe tocar Windows Defender para que deje pasar los binarios, los archivos del proyecto se generan localmente y no hay motivo para eso, y tampoco se debe apagar el broker con el kafka-server-stop.bat de la distribución porque usa wmic y devuelve cero aunque no haga nada.

15. Si algo falla y no se sabe dónde, se empieza por la prueba de humo, porque ordena las comprobaciones de menor a mayor dificultad y marca en qué punto se rompió.

Conclusiones

El laboratorio cerró con un pipeline Lambda funcional de punta a punta, desde la foto de los datos hasta la consulta final, y todas sus comprobaciones dieron el valor esperado.

La arquitectura se verificó a sí misma, el cruce de la capa de servicio puso las 111 horas del batch al lado de las 111 lecturas de los episodios del streaming y las dos dieron el mismo número, que es la tercera vez que ese conteo se confirmaba en la misma corrida.

La reproducibilidad quedó demostrada con hashes, la foto tiene su SHA-256 en el manifiesto, la ingesta tiene su hash estable, y las tres vistas de las capas de procesamiento produjeron firmas idénticas sobre el mismo tema.

La capa de velocidad respondió casi en vivo con una ventana de sesión que se ajustó al problema, juntando las ráfagas contaminadas y separando la calma, y la capa de lotes consolidó la semana completa sin ningún interruptor de limpieza.

El Docker opcional quedó validado de forma estática con trece comprobaciones sobre el archivo de composición, porque la máquina de referencia no trae Docker, y la imagen apache/kafka 4.1.2 quedó confirmada publicada en Docker Hub.

Los límites también quedaron escritos, un dispositivo con poca memoria obligó a un driver de Spark de 2 GB y carpetas de mezcla de 512 MB, el contenedor no corrió en vivo, y la foto de una semana es un tamaño que no pone a prueba la arquitectura a escala.

Aun con esos límites, el objetivo se cumplió, la misma pregunta de calidad del aire se respondió por dos caminos distintos y los dos llegaron al mismo número, que es exactamente lo que una arquitectura Lambda promete.

Bibliografía

Akidau, T., Bradshaw, R., C., Chernyak, S., Fernández-Moctezuma, R. J. Lax., R., McVeety, S., (2015). The dataflow model: A practical approach to balancing

correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. Proceedings of the VLDB Endowment, 8, 1792-1803.

<https://doi.org/10.14778/2824032.2824076>

Apache Software Foundation. (s. f.). Apache Kafka documentation.

<https://kafka.apache.org/documentation>

Apache Software Foundation. (s. f.) Apache Parquet. <https://parquet.apache.org/docs>

Apache Software Foundation. (s. f.) Kraft vs ZooKeeper. Apache Kafka 4.0

<https://kafka.apache.org/40/getting.started/zk2kraft>

Apache Software Foundation. (s. f.) Structured Streaming programming guide. Apache Spark <https://spark.apache.org/docs/latest/streaming/index.html>

Barcelona Supercomputing Center. (2022). CAMS European air quality forecasts, ENSEMBLE data. Copernicus Atmosphere Monitoring Service (CAMS) Atmosphere Data Store.

Kreps, J. (2014, 2 de julio). Questioning the Lambda architecture. O'Reilly

<https://www.oreilly.com/radar/questioning-the-lambda-architecture>

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. NetDB'11, Athens, Greek.

Marz, N., & Warren, J. (2015). Big Data: Principles and best practices of scalable realtime data systems. Manning Publications. <https://www.manning.com/books/big-data>

Ministerio del Medio Ambiente de Chile. (2011). Decreto Supremo N.º 12, que establece la Norma Primaria de Calidad Ambiental para Material Particulado Fino Respirable MP2,5. Biblioteca del Congreso Nacional de Chile.

<https://www.bcn.cl/leychile/navegar?idNorma=1025202>

National Institute of Standards and Technology. (2015). FIPS PUB 180-4: Secure hash standard (SHS). U.S. Department of Commerce.

<https://doi.org/10.6028/NIST.FIPS.180-4>

Open-Meteo. (s. f.). Air Quality API. <https://open-meteo.com/en/docs/air-quality-api>

SQLite. (s. f.). About SQLite. <https://sqlite.org/about.html>

World Health Organization. (2006). Air quality guidelines: Global update 2005: Particulate matter, ozone, nitrogen dioxide and sulfur dioxide. WHO Regional Office for Europe. <https://www.who.int/europe/publications/i/item/9289021926>

World Health Organization. (2021). WHO global air quality guidelines: Particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide.

<https://www.who.int/publications/i/item/9789240034228>

Anexo A. Glosario de términos

Concepto	Definición
Acks	Confirmación de escritura que pide el productor al broker, acá vale 1, o sea que basta con que el broker escriba en su propio log.
Batch	Procesamiento diferido que recorre el histórico completo de una sola vez, es el camino de la capa de lotes.
Broker	Es la confirmación de lectura de Kafka que recibe, guarda y reparte los mensajes, acá corre en local en el puerto 9092.
Capa de Lotes	Es el camino lento de la Arquitectura Lambda, lee el tema completo y deja las vistas consolidadas.
Capa de Servicio	Es el camino final que consulta las dos capas anteriores en SQLite y las cruza para responder.
Capa de velocidad	Es el camino rápido de la Arquitectura de Lambda, procesa los eventos apenas llegan con Spark Structured Streaming.
Checkpoint	Es la carpeta donde Spark guarda hasta dónde leyó, si se reenvía el tema hay que borrarlo con '--limpiar' para que no se crea que ya lo procesó.
Commit	Es una versión guardada del código en git, cada log imprime el HEAD del repositorio en el momento de la ejecución.
Consumidor	Es quien lee los mensajes de un topic, el verificador de la ingesta lee sin grupo de consumo para poder correr las veces que haga falta.
Cruce	Es la consulta final de la capa de servicio que pone horas del batch y lecturas de episodios en la misma fila y compara los totales.

Disparador (Trigger)	Regla que le dice a Spark cuándo lanzar lotes, acá se usa availableNow, que procesa todo lo disponible y termina solo.
Episodio	Es una racha de lecturas contaminadas seguidas separadas de la siguiente por un hueco mayor al espacio de sesión
Foto de datos o Imagen de datos	Descarga única de la semana de datos a un archivo en disco, es la fuente de verdad de todo el pipeline.
Hash / SHA-256	Huella digital de un archivo o de un conjunto de campos, dos corridas sobre los mismos datos deben producir los mismos hashes.
Kafka	Plataforma de eventos donde los productores publican en tópicos y los consumidores leen.
Kraft	Modo de gobierno propio de Kafka sin ZooKeeper.
Manifiesto	Es un archivo JSON que acompaña a la foto con la dirección usada, la fecha de descarga y el hash contenido.
Marca de Agua (Watermark)	Cota de retraso que le permite a Spark olvidar el estado de las sesiones que ya no van a crecer más, acá va en cuatro horas.
Modo Completo	Modo de salida del streaming que reescribe todas las sesiones vivas en cada lote, es el que va en la capa de velocidad.
Offset	Es un contador de posición dentro de una partición del topic, los consumidores arrancan desde el primero con startingOffsets earliest.
Parquet	Es un formato de archivo columnar que Spark relee sin parsear, es donde quedan las vistas de velocidad y de lotes.
Partición	Trozo paralelo de un topic, la clave de cada mensaje es la ciudad para que las lecturas de un mismo lugar caigan siempre en la misma.
Productor	Es quién escribe los mensajes en el topic, acá es un script en Python con la librería kafka-python.

PySpark	API de Python de Spark, con ella se escriben tanto las consultas batch como las de streaming.
Spark	Motor unificado de procesamiento que sirve para lotes y para flujos.
SQLite	Base de datos completa que vive en un solo archivo y viene con Python, se utiliza en el archivo aire.db de la capa de servicio.
Streaming	Procesamiento continuo de los eventos a medida que llegan, es el trabajo de la capa de velocidad.
Topic	Canal donde Kafka guarda los mensajes, en este laboratorio todo pasa por el topic lab03.eventos.
Umbral	Valor de 25 microgramos sobre metro cúbico sobre el cuál una hora se considera contaminada, viene del límite diario de la OMS y viaja dentro de cada mensaje.
Venv	Carpeta del intérprete y las librerías propias del proyecto dentro del repositorio, siempre se invoca con su ruta absoluta.
Ventana de sesión	Agrupación que junta los eventos que llegan seguidos y se cierra recién cuando pasa el tiempo de separación.
Winutils	Binario nativo que Windows necesita para que Spark pueda escribir archivos, su carpeta se declara con la variable de entorno HADOOP_HOME

Anexo B. Evidencias

Los logs, uno por comandos, dentro de la carpeta 'evidencias\logs\':

- '00_smoke_test', '01_descarga', '02_conector_kafka', '02_productor', '03_kafka_broker', '03_verificacion', '04_jars_locales', '04_kafka_cierre', '04_velocidad', '05_resumen_velocidad', '06_lotes', '07_servicio', y '08_verificacion_compose'.

Todos poseen extensión con '.log' que funciona como evidencia del funcionamiento.

Salidas de las capas en 'lambda/salidas/': el CSV de episodios con su Parquet, las vistas diarias y ciudades en Parquet y CSV, y la base 'aire.db' de la capa de servicio.